

Gaussian Quadrature in Lean and Rocq

Robert Joseph

California Institute of Technology

Abstract

We formalize Gaussian quadrature in Lean, starting from Appel and Bindel’s preliminary Rocq development and comparing the proofs needed to connect real analysis, floating-point arithmetic, and C execution. Using `mathlib`, `LeanPDE`, and `FloatLib`, we construct weighted Gaussian rules of every positive order on compact intervals, prove their remainder and convergence, and certify binary64 results for all ten rules stored in the example program. Checking the example exposes two incorrect error bounds: 0.00223 for two nodes and 0.02 for one node. We prove the replacements 0.00356 and 0.159, correct the remainder’s inconsistent node indexing, model the weight actually stored in C, and give counterexamples to three admitted helper lemmas. For the C proof, we replace the external cosine with a verified polynomial. For each stored rule, every permitted evaluation order of an initialized call then terminates with the certified result in our Lean adaptation of CompCert’s C semantics. We also prove that these calls preserve their results when translated to Clight. Separate application proofs reach formal assembly in Lean, while independent Rocq proofs use CompCert’s compiler theorem. Connecting the C library to those assembly proofs, proving general correctness of the source translation, and establishing agreement between the Lean and Rocq definitions of execution remain open.

1 Introduction

Appel and Bindel [2] study the verification of a C program that approximates

$$\mathcal{I}(f) = \int_{-1}^1 f(x) dx, \quad f(x) = \tfrac{1}{2}(1 - x) \cos x, \quad (1)$$

using Gaussian quadrature. The integrator occupies seven lines of C. Yet proving its accuracy requires real analysis, floating-point error bounds, memory and function-call reasoning, and a theorem about compilation. This makes it a useful example for comparing Lean and Rocq: the program is small enough to inspect, while its correctness depends on all of these proofs.

Their paper is titled *Formalization of Gaussian Quadrature and Application Verification (Preliminary Draft)*. Its admitted lemmas mark unfinished work. We review that preliminary version and the accompanying public source, distinguishing false statements from missing proofs and unassembled program modules. The source already contains work on correcting the derivative-bound hypotheses; we discuss its status in Section 2.5. Our results are in a separate development and do not complete the authors’ Rocq files.

Their Rocq development brings together `MathComp` and `MathComp-Analysis` [13, 1] for real analysis, `Flocq` [4] for floating-point arithmetic, `LAProof` [10] for rounding error in sums, `Interval` [14] for concrete inequalities, and `VST` [5] for C verification. CompCert’s compiler theorem [12] provides the next connection, from verified source behavior to formal assembly. Our development, `LeanQuadrature`, uses `mathlib` [17], `LeanPDE` [11], and `FloatLib` [8] for the mathematical and

numerical proofs. For execution, we adapt CLean [6] and prove the required properties directly from the language semantics.

The example also lets us check its accuracy claims independently of the program proof. The integral is $\sin 1$, and the exact two-point rule returns $\cos(1/\sqrt{3})$. Their difference is about 0.0035591571, which already exceeds the manuscript’s bound of 0.00223. The one-point bound is also false. We prove corrected bounds, identify inconsistent indexing in the admitted remainder theorem, and use the actual C weight where the original floating-point model differs. Section 2 gives the calculations and counterexamples. It also distinguishes three false admitted helper lemmas from unfinished program obligations, such as the missing callback body proof and module assembly. An admitted lemma is an assumption: checking a proof that uses it does not establish that assumption.

Our mathematical results apply beyond this integrand. For every positive order, we construct a Gaussian rule on a compact interval with a positive continuous weight. We prove exactness, positivity of the weights, the three-term recurrence, the integrated remainder, and convergence for continuous integrands. The original development supplies concrete root certificates through four nodes and leaves the general remainder admitted. Our floating-point results cover all ten rules stored in its C source, with error bounds for the actual constants and rounded operations. Appendix A compares all 24 numbered paragraphs of their mathematical presentation with the LeanPDE revision we use and the proofs added here.

At the C level, we replace the external cosine with a polynomial whose C body and numerical use are both verified. We prove that initialized calls to the integrator terminate with the certified result in every evaluation order permitted by our adapted semantics. We then prove that the six selected library functions preserve their call results when translated to Clight, CompCert’s simpler intermediate language for C. Separate proofs establish the results of ten application programs through formal assembly in Lean. We also prove those application results independently in Rocq, where they compose with CompCert’s compiler theorem.

These results leave a specific composition problem. The C proof covers the initialized library; the assembly proofs start from separately written Clight applications. We have not connected those applications to the translated library. General correctness of the source translation and a proof that the adapted Lean execution rules agree with Rocq’s also remain open. The comparison below separates the proved results from these remaining connections.

1.1 The five levels of verification

The five levels are the real integral, the exact quadrature rule, its floating-point model, the C implementation, and the compiled program. Four proofs connect adjacent levels. Write $\mathcal{Q}_n(f)$ for the exact quadrature sum and F for the binary64 result of the rounded functional model. The first two connections establish

$$|\mathcal{I}(f) - \mathcal{Q}_n(f)| \leq E_{\text{quad}}, \quad |\mathcal{Q}_n(f) - \text{real}(F)| \leq E_{\text{fp}}. \quad (2)$$

The first inequality bounds quadrature error over the reals. The second accounts for stored constants, callback error, and rounding. The triangle inequality then gives

$$|\mathcal{I}(f) - \text{real}(F)| \leq E_{\text{quad}} + E_{\text{fp}}.$$

The third proof shows that the C function returns F , with valid memory accesses and calls. The fourth preserves that source behavior through compilation. These last two proofs introduce no new numerical error; their task is to preserve the computation already analyzed.

The cosine implementation affects what can be concluded at the C level. The original program calls an external cosine, so its verification needs a contract for that implementation. Our replacement

polynomial has a verified C body and certified numerical results at the inputs used by the example. We also prove initialization of the stored tables. The resulting library theorems therefore need neither an assumed cosine contract nor an assumed table predicate.

Table 1 compares this work with Appel and Bindel’s development. The independent Rocq application proofs that we use with CompCert are described separately in Section 6; they are part of LeanQuadrature, not changes to the authors’ Rocq proofs.

Reading the labels. ✓ **Proved** means the stated theorem is checked under its hypotheses. 🟡 **Partial** and 🟡 **Conditional** mean an application still has obligations to discharge. ◦ **Open** means the connection is not proved. ✗ **Refuted** means a counterexample disproves the claim.

Level	Appel–Bindel in Rocq	LeanQuadrature in Lean
1. Real integral	MathComp, MathComp-Analysis: weighted real integral.	mathlib: interval integrals. LeanPDE: checked symbolic integration. ✓ Proved: the example integral equals $\sin 1$.
2. Numerical rule	MathComp, MathComp-Analysis. 🟡 Partial: recurrence and quadrature proofs, with root certificates through four points. The general remainder is admitted; convergence is unproved.	mathlib, LeanPDE, with our general quadrature proofs. ✓ Proved: construction at every positive order on compact intervals; exactness, positive weights, recurrence, the corrected remainder, and convergence.
3. Rounded model	Flocq, LAProof, Interval. 🟡 Partial: rounded fold and error analysis. The example’s accuracy argument uses admitted numerical helpers; two overflow helpers have false conclusions for permitted parameters.	FloatLib: binary64 arithmetic and rounding bounds. LeanPDE: root and weight enclosures. ✓ Proved: finite fold, error bounds, and ten stored rules; callback accuracy at the inputs used by the examples.
4. C execution	VST. ✓ Body proof: VST relates the integrator to its functional model under table and callback specifications. 🟡 Partial: the wrapper uses admitted numerical lemmas; the callback body and assembly into a Verified Software Unit (VSU), including global initialization, remain obligations.	Adapted CLean, FloatLib, with our execution proofs. ✓ Proved: initialized calls using the original functions and a verified C cosine polynomial, at ten orders and in every permitted evaluation order. Behavioral preservation to Clight for all six selected functions. ◦ Open: general source-translation correctness and agreement with Rocq’s execution semantics. The external-cosine variant retains its contract.
5. Compilation	CompCert. ✓ Compiler theorem: CompCert preserves program behavior. Using it for this application requires source correctness in its C semantics.	Adapted CLean, with our assembly proofs. ✓ Particular programs: direct proofs for ten polynomial applications at formal assembly. ◦ Open: a verified compiler, or a transfer of CompCert’s theorem into Lean.

Table 1: Libraries and proof coverage at each level. The Rocq column describes Appel and Bindel’s preliminary development, including the admitted dependencies discussed in Section 2. Our C proofs use a verified polynomial in place of the external cosine. The C library and assembly application proofs are separate results; a theorem connecting them remains open.

LeanPDE is a substantive dependency of these proofs. We reuse its quadrature interfaces, its two-point Gauss–Legendre rule and cubic exactness theorem, and its stability bounds under uniform approximation. Its certified root brackets, rational interval arithmetic, Taylor inequalities, and integration tactics establish the concrete enclosures. The same LeanPDE revision also contains

a sharp fourth-derivative error bound for the two-point rule, proved with a Peano kernel. That result was already available; our general remainder proof does not use it. We build the general weighted construction, integrated remainder, and convergence proof on mathlib. FloatLib supplies the binary64 operations and rounding theorems used to connect those real bounds to computation.

2 Corrections to the example and its proofs

Both the integral and the ideal quadrature sum have closed forms, so we can check the claimed errors directly. A quartic tests the remainder formula, and decoding the table literals tests whether C and the rounded model store the same values. Table 2 records these four findings. The rest of this section examines the admitted helpers and program specifications on which the application proof depends. These are corrections to a preliminary development. The absence of a proof is a coverage question; a counterexample establishes the stronger claim that a statement must change before it can be proved. Page references and source claims concern the examined preliminary manuscript and accompanying source [2, 3]; the repository records their exact versions and maps each finding to its declarations.

Location	Finding	Correction checked in Lean
Page 5, two points	✗ Refuted : the ideal error bound 0.00223.	✓ Proved : the replacement bound 0.00356. Actual error ≈ 0.0035591571 .
Page 5, one point	✗ Refuted : the ideal error bound 0.02.	✓ Proved : the replacement bound 0.159. Actual error $1 - \sin 1 \approx 0.1585290152$.
Page 4, remainder	✗ Refuted : the formula combines n nodes with a derivative of order $2n + 2$ and the polynomial p_{n+1} .	✓ Proved : the remainder uses $2n$ derivatives and p_n , with C^{2n} regularity. Two nodes require the fourth derivative.
Pages 6–7, weight	✓ Mismatch proved : the C and model weights have different binary64 bits; the C value exceeds the stated tolerance.	✓ Proved : the C value satisfies a $5 \cdot 10^{-16}$ weight bound. Our initialized library uses these actual C bits.

Table 2: Corrections and their proof status. The first three rows concern real mathematics. The fourth repairs the numerical model to match the stored C constant and uses it in the initialized Lean library.

2.1 The two-point error bound

Page 5 states the bound $223/100000 = 0.00223$ for the exact-real two-point rule. Symmetry in (1) gives

$$|\mathcal{I}(f) - \mathcal{Q}_2(f)| = |\sin 1 - \cos(1/\sqrt{3})| \approx 0.0035591571.$$

The error is larger than the proposed bound. The Lean proof works with rational enclosures for sine and cosine, so the displayed decimal is only an illustration. An upper enclosure for $\sin 1$ and a lower enclosure for $\cos(1/\sqrt{3})$ also give the corrected bound 0.00356. Listing 1 shows the final proof. The first rewrite replaces the integral and the sum by their closed forms, and the two remaining goals are linear inequalities between previously proved enclosures.

The C corollary on pages 1 and 11 uses the slightly larger bound 0.00224 for the reported result 0.83791182769499317. Its analytic premise therefore needs correction before the program argument can establish an accuracy bound. The counterexample above concerns the ideal rule. To

```

theorem corrected_two_point_bound :
  | (∫ x in (-1 : ℝ)..1, integrand x) -
    (integrand (-(1 / Real.sqrt 3)) + integrand (1 / Real.sqrt 3)) | ≤
    (356 / 100000 : ℝ) := by
  rw [integral_eq, two_point_eq, abs_of_nonneg]
  · linarith [sin_one_upper, cos_node_lower]
  · linarith [sin_one_lower, cos_node_upper]

```

Listing 1: The corrected two-point bound, from `Quadrature.Examples.TwoPointCosine`. The enclosure lemmas are proved earlier in the same module.

prove an error bound for a C execution, we must also account for its cosine implementation and rounded operations, as in Sections 4 and 5.

2.2 The one-point error bound

The one-point node is zero and its weight is two, so

$$Q_1(f) = 2f(0) = 1, \quad |\mathcal{I}(f) - Q_1(f)| = 1 - \sin 1 \approx 0.1585290152.$$

This exceeds the bound $2/100 = 0.02$ on page 5 by almost a factor of eight. We prove both the failure of that inequality and the replacement bound $159/1000 = 0.159$ in Lean.

2.3 Counting nodes and derivatives

The displayed `quadrature_error` statement on page 4 uses an n -node root collection together with a derivative of order $2n + 2$ and a polynomial indexed by $n + 1$. With n the number of nodes, the remainder requires order $2n$, as in Theorem 3.1. The source makes the node count explicit: its root certificate stores an n -tuple, and the quadrature sum ranges over $i < n$.

The formula $2n + 2$ is right for a rule with $n + 1$ nodes indexed from zero through n , which is Stewart’s convention [15]. The problem is that the node count and the derivative order follow different conventions in the same statement. For two nodes the required derivative is the *fourth*, and for one node it is the second.

A quartic makes the consequence concrete. For $f(x) = x^4$, two-point quadrature has error

$$\int_{-1}^1 x^4 dx - [(-1/\sqrt{3})^4 + (1/\sqrt{3})^4] = \frac{2}{5} - \frac{2}{9} = \frac{8}{45},$$

while $f^{(6)}$ is identically zero. The displayed formula would give zero error. Lean proves the exact error $8/45$ and the vanishing sixth derivative, which together refute that formula for a two-node rule.

The manuscript marks the general remainder theorem as admitted. Its source statement also quantifies over an arbitrary $f : \mathbb{R} \rightarrow \mathbb{R}$ with no smoothness premise. Our theorem, `GaussianRule.remainder`, uses the derivative of order $2n$ and requires C^{2n} regularity on the closed interval. Section 3 explains the proof.

2.4 A weight differs between C and the model

The C listing on page 6 stores the outer three-point weight as 0.5555555555555556. The functional-model value on page 7 is 0.5555555555555556. In binary64 these are different numbers:

C literal	0x3fe1c71c71c71c76
Functional-model literal	0x3fe1c71c71c71c72

The C value decodes exactly to 2501999792983611/4503599627370496. Its distance from the ideal weight 5/9 is

$$\frac{19}{40532396646334464} \approx 4.6876083262 \cdot 10^{-16},$$

which exceeds 2^{-53} . So the C constant misses the stated uniform half-ulp tolerance even when half an ulp is measured at one. Lean proves this inequality and the difference between the two stored values. We keep the original C constant and prove the uniform weight bound $5 \cdot 10^{-16}$ for the stored tables instead.

The mismatch also affects the program proof. The generated Clight program contains the C bits, but the table predicate in the VST specifications describes the model’s array. A body theorem may assume that predicate, but a proof from program initialization must establish it for the array that is actually there, and with these literals it cannot. We model the existing C literals and enlarge the error budget. Our selected library initializes those values and proves the required table predicate. The original VST development would need a corresponding change to its model and initialization proof. VST’s Verified Software Unit construction checks the connection between global initializers and the module’s data predicate. The quadrature source has not yet assembled this VSU, so the successful body proofs do not assert that initialization satisfies their table premise [19]. The middle three-point weight has different decimal spellings in the two files, but both spellings round to the same binary64 value, so that entry is not a second mismatch.

2.5 Assumptions in the original proofs

The two example lemmas, `error_1_0_1` and `error_1_0_2`, use `quadrature_error_bound_is_bound`, which depends on a Legendre specialization of the admitted `quadrature_error`. Interval checks an inequality derived from the assumed remainder. The remainder itself remains an assumption, so a later proof ending in `Qed` establishes an implication from that assumption. This explains how the final scripts can compile while the claimed error bound is false: the unchecked premise already contains the error.

The C development has a similar dependency. Its integrator body proof is complete under its specification. The example wrapper invokes admitted lemmas for function bounds, derivative bounds, callback accuracy, and quadrature error. The verification file supplies nine body lemmas for ten internal functions and never proves `body_testfun`. The wrapper obtains the callback contract from the global specification environment with `make_func_ptr`, which establishes what may be assumed about the callback, not that its C body meets the contract.

To see exactly what the completed proofs depend on, we rebuilt the original quadrature proofs and their LAMeasure dependencies and asked Rocq for the transitive assumptions of 43 declarations. The wrapper depends on ten of the twelve admitted declarations in the quadrature files, including the false positivity and overflow helpers described below. The integrator body depends on none of the twelve. The rebuild needed two explicit product-type annotations in the high-level 2D specifications, which resolve a notation ambiguity in our Rocq environment and change no proof, admitted lemma, or C code. All nine original body proofs then compile. The committed Clight

syntax was generated for AArch64 with the Apple ABI, and our review checks it under an x86-64 CompCert configuration.

The preliminary manuscript acknowledges the missing remainder proof. Consequently its concluding description of a first-principles argument should be read as the intended completed chain. The checked state of that chain still includes the assumptions above.

Work already under way on derivative bounds. The public source includes a separate `proof/g_max_deriv.v` that reformulates `fbound` and `deriv_bound` on a fixed interval with $a < b$. It requires both the starting and perturbed arguments to lie in that interval. Its target bounds the change in the function by a derivative bound times the displacement. This is the appropriate mean-value argument for controlling perturbed quadrature nodes. The file is work in progress: `g_max_deriv` ends in `Qed` but uses the admitted `g_max_deriv_oneway`, and other admissions remain. It is separate from the integrated quadrature proofs in the examined revision [3]. This ongoing correction is relevant to the example’s admitted function and derivative bounds. Our accuracy theorem states the required Lipschitz bound explicitly, and the concrete numerical certificates establish their own bounds.

A positive function with zero integral. The admitted lemma `Rintegral_gt_0` assumes continuity, nonnegativity, and a nonzero value somewhere in $[a, b]$, and concludes that the integral is positive. The missing condition is $a < b$. For $a = b = 0$ and $h(x) = 1$, all three premises hold, but

$$\int_{[0,0]} h(x) dx = 0$$

because a singleton has Lebesgue measure zero. Our Lean counterexample checks the three premises and refutes the conclusion. We also check the example in Rocq with MathComp-Analysis, and a separate diagnostic applies the original admitted lemma to those premises and derives a contradiction. The source uses this helper in its polynomial-positivity argument, inside a section that already assumes $a < b$, so adding the missing premise is a natural repair for that use. Our `polynomialIntegral_square_pos` requires $a < b$ and proves the result using `mathlib`.

A specification that names the wrong function. The source also includes Hughes quadrature routines for a triangle. Its `hughes_weight_spec` begins with `DECLARE _gauss2d_weight` where the intended identifier is `_hughes_weight`. As a result, `quadrules_ASI` contains two specifications named `_gauss2d_weight` and none named `_hughes_weight`. The body proof still checks because VST’s `semax_body` receives a function body and an identifier–contract pair, checks the body against the contract, and discards the identifier. Associating names, bodies, and contracts is a separate obligation when the verified interface is assembled. A VSU checks that the specification used at a call is supplied by the corresponding body or by a contract that implies it [19]. That is the missing integration step here. The identifier can be corrected without changing the contract, and the existing body theorem then proves the retagged specification. This is a defect in the specification of the triangle routine, not a counterexample to its numerical result.

2.6 Two overflow lemmas need stronger hypotheses

The floating-point proof must show that every intermediate result is finite. The source introduces a condition named `parameter_limits` and leaves two helper inequalities admitted. Neither helper follows from the stated conditions for all permitted parameters, and the problem can be seen without running any floating-point code.

Specialize to binary64 and write

$$u = 2^{-53}, \quad \eta = 2^{-1075}, \quad T = 2^{1024}.$$

These are the relative-error constant, the absolute-error constant, and `fmax` in the version of LAPProof used by the source. The largest finite binary64 number is smaller than T . Take one node, the zero integrand, and a callback that always returns zero. Choose the valid but loose function bound $F = T/4$ and set the derivative and callback-error bounds to zero. The node zero and the weight two are represented exactly. The source’s range condition becomes

$$\left(1 + \frac{u}{1+u}\right)(2+u)\frac{T}{4} + \eta < T,$$

which holds, and its earlier product helper also holds. But the admitted `finite_integrate_model_aux` would require

$$\frac{T}{2} + \frac{T}{4}(u^2 + 3u) + \eta < \frac{T}{2(1+u)}. \quad (3)$$

The left side is greater than $T/2$ and the right side is smaller. This is a counterexample at a positive order, with valid bounds for both the mathematical function and the callback.

The other helper, `gauss_pt_wt_limit_aux`, fails at zero nodes, which the source’s type `'I_5` permits. With function bound $F = T$ and zero callback error, `parameter_limits` reduces to $0 < T$, while the helper requires $2T(1+u) + \eta < T$. Either the empty reduction needs separate treatment or the helper needs a positive-order premise.

Lean checks both counterexamples in `Quadrature.Examples.Overflow` with exact arithmetic on the transcribed inequalities. Both callbacks return zero, and their actual computations stay finite. The contradictions concern the proposed sufficient conditions for the helper lemmas. LAPProof’s summation theorem has its own valid precondition; the missing step is to derive it inside the quadrature development. Our finiteness proof, in Section 4, bounds the sum of absolute products together with the accumulated rounding errors.

3 Gaussian rules in Lean

The remainder theorem depends on the nodes, weights, and polynomial exactness of the rule. Appel and Bindel follow Stewart [15]: they construct the monic orthogonal polynomials by the three-term recurrence and take a `roots_of_ortho_p n` certificate containing n ordered roots, which their concrete Legendre instances supply through four points. We construct the orthogonal polynomials by orthogonal projection, prove the existence of their roots by a counting argument, and derive the recurrence afterward. This yields a Gaussian rule for every positive order. The construction uses `mathlib`’s polynomial algebra, orthogonal projection in finite-dimensional inner product spaces, and Lagrange interpolation. Its remainder and convergence proofs use Rolle’s theorem and uniform polynomial approximation. The paragraph-by-paragraph comparison in Appendix A separates these additions from LeanPDE’s existing two-point development.

Fix $a < b$ and a continuous function $w : [a, b] \rightarrow \mathbb{R}$ with $w(x) > 0$. The intended integral is

$$\mathcal{I}_w(f) = \int_a^b f(x)w(x) dx.$$

For a positive integer n , a Gaussian rule consists of distinct interior nodes x_i and weights λ_i , with $\mathcal{Q}_n(f) = \sum_{i=1}^n \lambda_i f(x_i)$. Throughout the paper n is the *number of nodes*.

3.1 Choosing the nodes and weights

The integral defines an inner product on real polynomials, $\langle p, q \rangle = \mathcal{I}_w(pq)$. It is positive definite because a nonzero polynomial cannot vanish on a nondegenerate interval, so its square has positive integral against w . Orthogonal projection onto the polynomials of degree less than n constructs the unique monic polynomial p_n of degree n orthogonal to that space.

The root argument shows that p_n has exactly n distinct roots in (a, b) , and these are the nodes. Lagrange interpolation defines the weights. Polynomial division by p_n then proves exactness:

$$\mathcal{Q}_n(q) = \mathcal{I}_w(q) \quad \text{whenever} \quad \deg q \leq 2n - 1. \quad (4)$$

Writing $q = p_n s + r$ gives $\deg s < n$ and $\deg r < n$. Orthogonality kills the integral of $p_n s$, and evaluation at the nodes kills its quadrature sum.

If ℓ_i is the i th Lagrange basis polynomial, exactness applies to ℓ_i^2 and gives $\lambda_i = \mathcal{I}_w(\ell_i^2) > 0$. Applying exactness to the constant polynomial gives $\sum_i \lambda_i = \mathcal{I}_w(1)$. These two facts control perturbations of the integrand later without estimating each weight separately. The development also proves the three-term recurrence, including the positivity of the norm ratio, and identifies the explicitly represented low-order Legendre rules with the constructed monic family.

3.2 Integrating the remainder

Theorem 3.1 (Integrated Gaussian remainder). *Let $a < b$, let w be positive and continuous on $[a, b]$, let $n > 0$, and let $f \in C^{2n}([a, b])$. For the Gaussian rule constructed above, there exists $\xi \in [a, b]$ such that*

$$\mathcal{I}_w(f) - \mathcal{Q}_n(f) = \frac{f^{(2n)}(\xi)}{(2n)!} \int_a^b p_n(x)^2 w(x) dx. \quad (5)$$

In Lean the smoothness hypothesis is `ContDiffOn` on the closed interval, and the derivative is the iterated derivative within that interval, so we never assume a smooth extension beyond the integration domain. More precisely, the proof takes functions $F_0, \dots, F_{2n}$ continuous on $[a, b]$, with $F'_k = F_{k+1}$ on (a, b) for $k < 2n$. The `ContDiffOn` version supplies this chain using `iteratedDerivWithin`.

Taylor–Lagrange itself is already in the mathlib revision used here: `taylor_mean_remainder_lagrange` gives the Taylor remainder about one center. Our added interpolation theorem concerns n distinct nodes, matching the function and its first derivative at each node. It is this Hermite remainder, followed by an integral argument, that proves Theorem 3.1. We do not formalize arbitrary prescribed derivative multiplicities in the interpolant.

Let H be the Hermite interpolant of degree below $2n$ matching both f and f' at the nodes. For a point x that is not a node, choose $c = (f(x) - H(x))/p_n(x)^2$ and consider

$$G(y) = f(y) - H(y) - c p_n(y)^2.$$

There are double zeros at the n nodes and one more zero at x . Repeated Rolle supplies a zero of the $2n$ th derivative. Since $H^{(2n)} = 0$ and $(p_n^2)^{(2n)} = (2n)!$, this identifies c . At a node the pointwise remainder is zero directly. This pointwise theorem needs continuity only for F_k with $k < 2n$, together with the derivative chain in the interior. The integrated theorem additionally uses continuity of F_{2n} .

The pointwise result does not yet give the integrated remainder. Its witness ξ_x varies with x , and nothing says that $x \mapsto \xi_x$ is measurable, let alone continuous. Integrating an arbitrarily chosen witness would introduce an unjustified regularity assumption. We use its range instead. Put

$$r(x) = (f(x) - H(x))w(x), \quad \phi(x) = \frac{p_n(x)^2 w(x)}{(2n)!}, \quad g = f^{(2n)}.$$

For every x , $r(x)$ equals $g(y)\phi(x)$ for some $y \in [a, b]$. The function ϕ is nonnegative with positive integral, so the minimum and maximum of the continuous function g bound $\int r / \int \phi$, and the intermediate value theorem supplies a point where g equals that quotient. Exactness for H now gives (5). The argument depends only on the range of g . We state it as a separate integral theorem so that other remainder proofs can reuse it.

If $|f^{(2n)}| \leq M$ on the interval, the absolute error is at most $M\mathcal{I}_w(p_n^2)/(2n)!$. For unit weight on $[-1, 1]$, the coefficients of M at orders one through four are $1/3$, $1/135$, $1/15750$, and $1/3472875$.

3.3 Convergence

For a continuous f , choose a polynomial q uniformly close to f . Once the order is high enough to integrate q exactly, positivity and the constant total weight give

$$|\mathcal{I}_w(f) - \mathcal{Q}_n(f)| \leq 2\mathcal{I}_w(1) \|f - q\|_\infty.$$

Uniform polynomial approximation proves convergence. Stewart lists this as a consequence of positive weights, and the manuscript notes it as unproved because the applications did not need it. The result does not require differentiability and is distinct from the derivative-based rate in Theorem 3.1.

The program stores rules for orders one through ten. For each table we identify the ideal nodes and weights, prove enclosures for them, and bound the distance from the stored constants. These certificates let the general theory apply to the concrete program. Everything here is on a compact interval. Gauss–Hermite and Gauss–Laguerre rules would need integrability and approximation arguments on unbounded domains. They are outside our development and explicitly outside the implementation described in the preliminary paper.

4 Floating point in Lean

The integral bound concerns exact nodes and exact arithmetic. To apply it to a program we need a second model that fixes the stored values, the callback, and the order of rounding. The original Rocq model is a left fold of rounded products and sums, parameterized by a floating-point format and a NaN policy, and its analysis uses Flocq, LAPProof’s dot-product results, and Interval. The Lean model uses the same evaluation order with FloatLib’s binary64 operations [8]. FloatLib is executable, so a statement about a particular rounded computation can often be settled by kernel evaluation, and it comes with rounding bounds for the general argument. Let $\hat{x}_i, \hat{\lambda}_i$ be stored nodes and weights, and let $\hat{f}$ be a floating-point callback. The computation is

$$s_0 = +0, \quad t_i = \text{fl}(\hat{\lambda}_i \hat{f}(\hat{x}_i)), \quad s_i = \text{fl}(s_{i-1} + t_i). \quad (6)$$

Each multiplication and addition rounds separately. Reassociating the sum or using fused operations would change the computation being verified.

A stored-rule certificate states that every node and weight is finite, the decoded nodes lie in $[-1, 1]$, and

$$|\text{real}(\hat{x}_i) - x_i| \leq \delta_x, \quad |\text{real}(\hat{\lambda}_i) - \lambda_i| \leq \delta_w.$$

For the ten stored tables, common bounds are $\delta_x = 6 \cdot 10^{-16}$ and $\delta_w = 5 \cdot 10^{-16}$. The latter covers the actual C literals, including the three-point constant from Section 2.

Theorem 4.1 (Accuracy of the rounded rule). *Let $n > 0$ and let a stored rule certify an ideal Gauss–Legendre rule. Suppose $|f| \leq B$, f is L -Lipschitz, $f \in C^{2n}([-1, 1])$, and $|f^{(2n)}| \leq M$. Assume*

$B, e, L \geq 0$ and that, on every finite binary64 input decoding into $[-1, 1]$, the callback returns a finite value with error at most e from f at that decoded input.

Write ε_R for FloatLib’s local absolute rounding bound at radius R . If

$$(2 + n\delta_w)(B + e) + 2n\varepsilon_R \leq R \leq \text{maxFinite}, \quad (7)$$

then the execution in (6) is finite, and

$$\begin{aligned} |\text{real}(s_n) - \mathcal{I}(f)| &\leq 2n\varepsilon_R + (2 + n\delta_w)(e + L\delta_x) + n\delta_w B \\ &\quad + \frac{M}{(2n)!} \int_{-1}^1 p_n(x)^2 dx. \end{aligned} \quad (8)$$

Each term in (8) has a distinct source. There are n products and n additions, each contributing at most ε_R . At a stored node, callback error and displacement of the node contribute at most $e + L\delta_x$, and multiplying by the total absolute stored weight gives the factor $2 + n\delta_w$. Changing the weights themselves contributes $n\delta_w B$. The last term is the exact-real quadrature error from Theorem 3.1.

The range condition (7) is what makes this a statement about finite arithmetic. It bounds the absolute mass of all products and reserves room for every rounding step. The proof establishes finiteness and the rounding estimate together. This supplies the finiteness argument for which the original development uses its admitted helpers. The resulting budget can then be discharged for each concrete application.

The callback hypothesis in Theorem 4.1 covers the whole input interval. For a fixed rule we can also prove accuracy directly at the finitely many inputs the computation uses. We do this for FloatLib’s executable cosine at the two stored nodes, with cosine error at most 10^{-15} , by evaluating the implementation in the kernel and comparing with a rational Taylor enclosure. The resulting rounded quadrature value has absolute integral error at most 0.00356. This covers FloatLib’s cosine at these inputs. A platform `libm` would need its own contract.

For the ten application proofs we use a callback that evaluates the degree-14 Taylor polynomial for cosine by a fixed rounded Horner scheme, so that no external call remains, and we certify each final result against a rational enclosure of $\sin 1$. Section 5 explains why this choice lets us discharge the callback contract in C.

5 Verifying C execution in Lean

The numerical proof bounds the error of a value F . We must now show that the C integrator returns that value. Listing 2 contains the complete function. The proof has to justify each table access and function call and show that the compound assignment performs the same rounded multiplication and addition as (6).

After k iterations, the loop invariant relates the C variable `s` to the model’s partial sum s_k , and it retains the table facts and callback specification needed for the next iteration. At loop exit $k = n$ and $s_n = F$, which connects the numerical theorem to the returned value. Appel and Bindel express this argument in VST’s `semax_body` judgment, and VST’s soundness theorem relates that judgment to CompCert’s semantics [18, 5]. Our Lean proofs work directly with execution rules, using the loop invariant in proofs about the permitted transitions.

CLean [6] is an experimental fork of CompCert whose `clightgen` can emit Clight syntax as Lean, together with a port of the Clight semantics. We adapted it in two ways. The exported semantics used host floating-point operations. We replaced these with FloatLib’s definitions so

```

double integrate (double (*f)(double), int n) {
  int i;
  double s = 0.0;
  for (i=0; i<n; i++)
    s += gauss_weight(i,n) * f(gauss_point(i,n));
  return s;
}

```

Listing 2: The original one-dimensional C integrator. The table accessors and callback are verified separately.

that arithmetic certificates can be checked by kernel evaluation. External calls were axioms, and we replaced them with explicit environment parameters and determinism contracts, so that every external assumption appears in the theorem statement. These definitions support the execution proofs throughout the development. We also formalized CompCert’s typed C expression reductions and small-step machine in Lean, using the same value and memory definitions. The rules permit every operand order allowed by CompCert’s C semantics. This is a manual adaptation: a theorem relating its definitions to the original Rocq definitions remains open.

The safety conclusion is relative to these execution rules, including their treatment of unsafe expressions and invalid memory accesses. A proof in this model alone does not establish that a manual adaptation has preserved every relevant C behavior. The semantic correspondence remains an obligation alongside the application proofs.

Lean also has a reusable separation-logic framework. Iris-Lean [7] provides the Iris base logic, a language-parametric program logic, and an interactive proof mode. The paper demonstrates verification and adequacy for HeapLang. Using this framework for our C library would require an instance for the C/Clight semantics, verified rules for its memory operations and calls, and an application of the adequacy theorem to those executions. Our development does not use Iris-Lean; we have not supplied that integration. In particular, direct execution proofs for these functions do not provide VST’s support for separately verified modules, higher-order function-pointer specifications, or concurrency. A C program can implement its own object system with function pointers and hidden mutable state. Scaling to such programs calls for compositional specifications and proof automation, beyond the operational semantics and application lemmas proved here.

5.1 Calls to the original C functions

We parse the original C text and select five functions: `gauss_point`, `gauss_weight`, `integrate`, `testfun`, and `integrate_testfun`. Lean checks equalities that identify their parsed and typed syntax and the 110 table initializers with the independently imported Clight records. The parser handles this selected part of the source and does not interpret the includes or the unrelated two-dimensional functions.

The generic call proofs separate what the integrator establishes from what its caller must supply. Each complete call includes allocation, parameter binding, expression evaluation, table loads, the loop, and return with frame release. The callback proof follows its C body, so the external-cosine variant retains a contract for cosine.

For the two-point wrapper, write $P(m)$ for the required table cells, function lookups, and

cosine-call contracts in the initial memory m . The theorem has the shape

$$P(m) \implies \exists m', v. \text{Call}(\text{integrate_testfun}, m, m', v) \wedge \text{Finite}(v) \wedge |\text{real}(v) - \sin 1| \leq 0.00356. \quad (9)$$

The conclusion also identifies v with the floating-point fold and states which memory properties the call preserves. Global initialization and the chosen cosine implementation must establish $P(m)$. The polynomial library below supplies those proofs. Equation (9) describes the existence of a terminating strategy execution. Our total-correctness proof establishes the stronger property that every permitted small-step execution terminates with the specified result.

5.2 Replacing the external cosine call

The replacement starts from the degree-14 Taylor polynomial

$$P_{\cos}(x) = \sum_{k=0}^7 \frac{(-1)^k x^{2k}}{(2k)!}.$$

The C function stores binary64 approximations to its eight coefficients. It first assigns the rounded square of the argument to $\mathbf{x}$, then evaluates the polynomial in that value by Horner’s rule. Lean checks the decimal literals and the complete parsed body. The execution proof covers allocation, parameter binding, the assignment, every rounded product and sum, the return cast, and freeing. For every binary64 input, the C function returns exactly the value of the corresponding floating-point model and preserves the caller’s memory. The numerical certificates establish the approximation accuracy needed at the inputs used by the quadrature example.

The library constructor reads the five original functions and both tables from the retained C text, and reads `cos` from the replacement file. It installs six function definitions, allocates and initializes the two tables, and proves that every function lookup and table load required by the call proofs succeeds. The wrapper theorem therefore has no table or cosine contract left as a hypothesis. Starting with these source texts, it constructs the library and an initialized call returning the finite value `0x3fead02c771c35ed`, whose distance from $\sin 1$ is at most 0.00356. The integrator theorem gives the corresponding initialized calls and error bounds for every stored order from one to ten. Both the wrapper and integrator theorems cover every permitted evaluation order.

5.3 Why the result holds in every evaluation order

In the loop body, C may evaluate the weight accessor before or after the node accessor and callback. The assignment’s location can also be computed before or after its right-hand side. A proof that chooses one order does not establish what happens in the others. Our small-step semantics retains all these choices, including an explicit transition to a stuck state when an exposed subexpression is unsafe.

We define a total-correctness judgment with two cases. A state already satisfies the postcondition, or it has a successor and every permitted successor satisfies the same judgment. Each step must be silent for these library calls. The definition is inductive, so an infinite chain of steps cannot satisfy it. The successor requirement excludes getting stuck. At the library boundary the postcondition is a terminal return with the specified value and preserved caller memory.

The central memory fact is that nested calls leave the caller’s existing loads and permissions unchanged. They may allocate and free their own local blocks. Different operand orders can assign different block numbers to those temporary allocations, so the proof does not require identical

final heaps. It requires the same returned value and the same preservation property. Table reads and callback arguments therefore retain their values while another operand calls a function. The compound assignment updates only the accumulator, and the postincrement updates only the loop index. The loop invariant then gives the same rounded left fold in every order.

Theorem 5.1 (Total correctness of the initialized C library). *Let $1 \leq n \leq 10$, and let F_n be the binary64 fold of the stored n -point rule with the verified polynomial callback. From the library’s initialized memory, every execution of the original integrator in the adapted C small-step semantics terminates, returns F_n , and preserves the caller’s memory. The result is finite and satisfies the error bound in Table 3. The original two-point wrapper returns `0x3fead02c771c35ed` with integral error at most 0.00356.*

The theorem has no table or cosine-call hypothesis. Lean checks the source selection and initialization as part of its application.

The shared total-correctness lemmas handle pure expressions, calls, assignments, and control flow; the function proofs supply the table facts and loop invariant. This gives us a reusable basis for direct proofs. VST supplies a much larger program logic and automation library. A C-specific interface built on a framework such as Iris-Lean could reduce the work of verifying further functions.

5.4 Preservation under C-to-Clight normalization

The frontend moves C parameters and locals into Clight temporaries and separates nested calls into statements. We prove that this translation preserves the selected library calls. Both outputs come from the same two source files, including the replacement cosine. In particular, the translated cosine is the actual frontend output: it keeps the source’s assignment to `x`, signed coefficients, and rounded Horner expression.

The proof uses the common functional result. The C total-correctness theorem covers every operand order; a separate Clight execution proof reaches the same value. All functions in the normalized library are internal. An invariant keeps every reached state in this internal fragment, where transitions are silent and deterministic under any interpretation of external calls. The finite Clight execution therefore determines every execution, without an external-determinism assumption.

Theorem 5.2 (Preservation of the selected C calls). *Let C and L be the typed C and normalized Clight libraries obtained from the retained quadrature source and the shipped cosine. Both initializers succeed with the same memory. For every stored accessor cell, every binary64 argument to the callback or cosine, every integrator call with the initialized callback and $0 \leq n \leq 10$, and the original two-point wrapper,*

$$\text{Returns}_C(\text{name}, \text{args}, t, v) \iff \text{Returns}_L(\text{name}, \text{args}, t, v).$$

Here a return includes initialization, lookup of the named function, and execution to a return state with trace t and value v . Both calls terminate without getting stuck. Their returned memories agree on every load and permission on blocks allocated before the call.

For the wrapper, both sets of returns contain exactly the empty trace and the binary64 value `0x3fead02c771c35ed`. Equality of their entire final memories would be too strong: C allocates and frees parameter and local blocks, while Clight uses temporaries. The memory relation preserves what the caller could already access.

The theorem covers the selected calls in the two translated Lean representations. A general theorem about the source translator would have to justify parsing and elaboration against an

independent definition of C source meaning, including preprocessing and local static arrays. That theorem remains open, as does agreement between these Lean execution rules and their Rocq counterparts.

6 Checking compiled applications

CompCert can compile a supported C program in place of GCC, and its theorem says how the behavior of the compiled program relates to that of the source. To use it for quadrature accuracy there must first be a theorem saying that the source computation is accurate, and that theorem must describe C in the semantics consumed by CompCert’s theorem. The verified core begins with formal C syntax after parsing and elaboration and ends at formal assembly syntax [16]. Schematically,

$$\underbrace{C \models S}_{\text{source correctness}} \quad \underbrace{\text{compile}(C) = A \implies \text{Beh}(A) \preceq \text{Beh}(C)}_{\text{compiler preservation}}.$$

Here S is the numerical specification and $\preceq$ is the behavior relation used by the compiler theorem. Our Lean numerical theorem concerns FloatLib’s function F , while CompCert’s theorem concerns a C program C in Rocq. A source proof must identify the computation of C with F . Reusing that proof across Lean and Rocq also requires agreement about the values, states, and propositions it describes. Using CompCert discharges the compilation step once source correctness is available in the matching semantics.

We pursue two approaches: direct proofs of the compiled applications in Lean, and application proofs in Rocq that use CompCert’s general compiler theorem. Figure 1 shows where they connect to the numerical proofs and where a connection is still missing.

6.1 Proving the compiled programs directly in Lean

For the direct Lean proofs, we ran the ten polynomial applications through CompCert and exported selected intermediate programs: Cminor, CminorSel, the initial RTL and the RTL immediately before register allocation, LTL, Linear, Mach, and formal x86-64 assembly. Each exported program is a Lean term. For each intermediate language we wrote the transition rules and an executable checker over the same FloatLib-adapted value and memory models used for Clight. We then proved, for each of the ten programs at each stage, that execution from the initial state terminates with the certified result annotation and exit status zero. Every reachable state can finish, and no execution runs forever or gets stuck.

At formal assembly an order- n run takes $153n + 37$ semantic steps. The Mach proofs check the generated return addresses against the translated continuations at all sixteen call sites. We also relate selected parts of adjacent stages, such as the contents and permissions of the global memory blocks in Clight and Cminor.

The result is a proof of each imported program’s behavior in the Lean semantics. A general compiler theorem would have to relate the input and output of every successful compilation. To reuse CompCert’s theorem in Lean, we would need to relate the exported syntax and adapted semantics to their Rocq definitions, then justify interpreting the Rocq theorem in Lean. Another approach would be a translation validator that checks each compilation and has a proved soundness theorem.

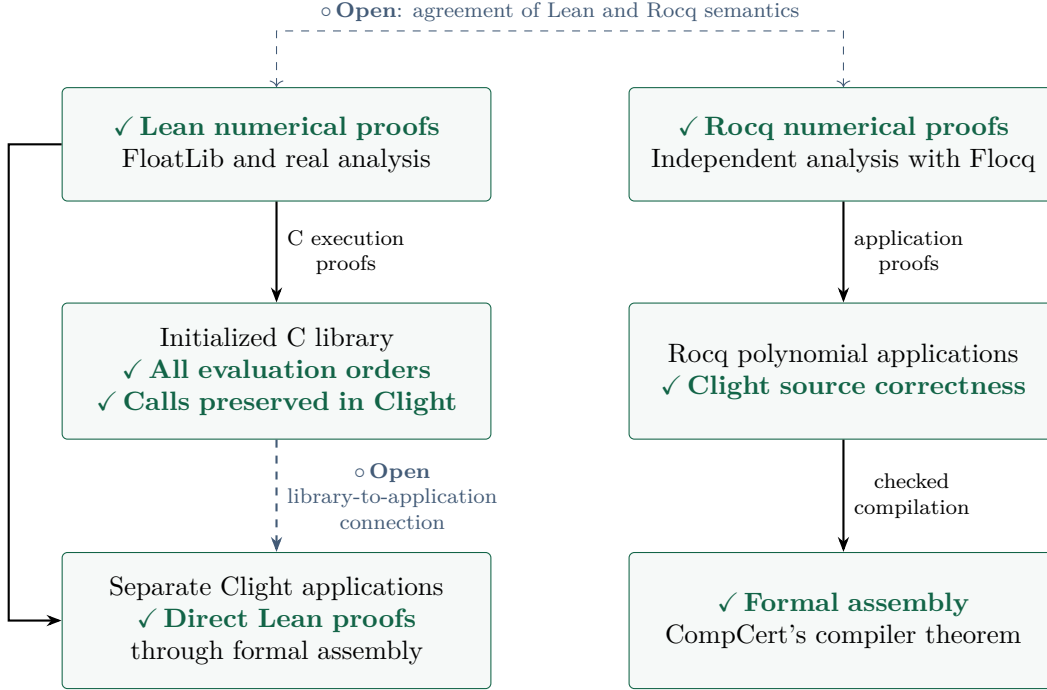


Figure 1: The two developments within LeanQuadrature. Solid arrows connect proved results; dashed arrows mark missing connections. Lean numerical proofs support both the initialized C library and the separate applications checked through assembly. The right column describes our independent Rocq proofs. Both columns end at formal assembly syntax, before native assembly and linking.

6.2 Using CompCert in Rocq

The second route uses CompCert’s theorem in Rocq. We reproved the numerical results for the polynomial programs in Rocq, independently of the Lean proofs, and composed them with compilation and termination proofs. All ten orders have integral-error certificates whose conclusions hold for every behavior of the compiled program. Successful compilation is itself checked inside Rocq, so it does not remain a hypothesis.

Evaluating CompCert inside the Rocq kernel needs a configuration that the kernel can run. We supply executable definitions for the compiler’s external choices, disable selected optimizations and inlining, and use a finite set of register-allocation candidates that CompCert’s validator must accept. The configuration also exposes computations needed by evaluation and uses a bounded iteration procedure whose invariant theorem we reprove. The certificate script rebuilds the compiler proofs from source. Its finite allocator may fail on other programs, so the results describe this checked compiler variant and these applications.

6.3 Connecting the two proof systems

The independent Rocq proofs supply the numerical premises for the CompCert applications. Reusing the Lean proof would avoid repeating that analysis. One route is a sound interpretation or translation of the relevant arithmetic, program states, and propositions. Another is a common certificate format with sound checkers in both systems. Either approach must justify the interpretation of the checked result in the receiving system.

Floating-point arithmetic shows what has to be related. The FloatLib and Flocq models used here have different NaN policies, and paired certificates show that unrestricted bitwise equivalence between them is false. For quadrature the useful comparison concerns finite computations and must preserve the sign of zero. We have proofs in both systems that characterize rounding through integer quotients, remainders, parity, and exponent alignment. Relating those integer representations is still open.

6.4 From formal assembly to an executable

The CompCert manual places assembly printing, assembling, and linking outside the verified core. Our final property concerns a `quadrature-result` semantic annotation and exit status zero in formal assembly. The printer turns the annotation into a comment, so this property does not yet describe a printed answer from a native executable. Connecting it to one requires proofs about the intended output operation and the remaining translation steps. CompCert could also compile C emitted by Lean’s own compiler. Its theorem would then cover the C compilation step, and the Lean-to-C translation and runtime would still need justification.

7 Results

Table 3 gives the results for the ten polynomial applications. Each row identifies the binary64 answer and a proved bound on its distance from $\sin 1$. Lean and our Rocq development check these bounds independently. At the higher orders the bounds are on the scale of rounding and stored-table perturbations, and increasing the number of nodes does not make these finite-precision bounds decrease at every step: the seven-node bound is smaller than the eight-node bound.

Nodes	Binary64 result bits	Error at most	Status
1	3ff0000000000000	0.159	✓ Proved
2	3fead02c771c35ed	0.00356	✓ Proved
3	3feaed9520c8a014	$3.1 \cdot 10^{-5}$	✓ Proved
4	3feaed5443a06327	$1.5 \cdot 10^{-7}$	✓ Proved
5	3feaed548f3f6f46	$4 \cdot 10^{-10}$	✓ Proved
6	3feaed548f08f234	$8 \cdot 10^{-13}$	✓ Proved
7	3feaed548f090ce1	$2 \cdot 10^{-15}$	✓ Proved
8	3feaed548f090cce	$4 \cdot 10^{-15}$	✓ Proved
9	3feaed548f090ccf	$4 \cdot 10^{-15}$	✓ Proved
10	3feaed548f090cd4	$3 \cdot 10^{-15}$	✓ Proved

Table 3: Proved error bounds for the ten initialized polynomial applications. The callback evaluates the degree-14 cosine polynomial with fixed rounding at each operation.

LeanQuadrature [9] contains the proofs, the build instructions, and a map from every result in this paper to its theorem declaration. The full Lean build and transitive assumption checks cover the project theorems, their private helpers, and the adapted CLean library, and the only permitted axioms are `propext`, `Classical.choice`, and `Quot.sound`. The Rocq compilation proofs have their own record of assumptions, including the logical and external-semantics dependencies of CompCert and Flocq. The project contains roughly 53,000 lines of Lean, a large share of it imported program syntax, and about 10,000 lines of Rocq. LeanPDE is a private dependency. The public source

distribution does not include it; readers who wish to reproduce the build should request access from the author.

8 Discussion

The mathematical and numerical proofs draw heavily on existing Lean libraries. `mathlib` provides the analysis needed for the general Gaussian construction; `LeanPDE` contributes quadrature interfaces, concrete rules, and certified symbolic calculation; `FloatLib` makes the binary64 computation executable inside the logic. The new work builds on these libraries to prove the weighted construction, remainder, convergence, and the accuracy of the ten stored rules. `LeanPDE` is essential to this implementation because the proofs reuse those definitions and theorems. Its existing sharp two-point error bound must be distinguished from our general remainder theorem. `TorchLean` is a transitive dependency of `LeanPDE`; we do not use its tensor or neural-network interfaces.

The larger difference between the two systems appears in program verification. `Rocq` supplies VST’s C program logic and `CompCert`’s compiler theorem in the same proof system. Our Lean development supplies direct execution proofs for this library and these compiled applications. `Iris-Lean` provides a reusable program-logic foundation, but applying it here would require a C/Clight language instance and verified rules for memory operations and calls, together with practical automation. The current application proofs do not establish that direct execution reasoning alone will scale to large modular C programs.

The source review gives concrete corrections to Appel and Bindel’s preliminary application argument. The two error bounds and three admitted helpers have counterexamples; the remainder needs consistent indexing and a smoothness premise; the table model must agree with the initialized C values. The callback body proof and the incorrectly named Hughes specification are separate program obligations. Our Lean proofs supply corrected mathematical statements and verify the polynomial replacement. The public derivative-bound work already moves toward stronger hypotheses, and the missing quadrature VSU explains why the initializer and function-identifier mismatches are not caught by the existing body proofs. These are unfinished steps in the authors’ development, not failures of VST’s program logic. The overflow inequalities were transcribed into Lean and checked against the source by inspection. The positivity counterexample was also checked against the original statement in `Rocq`.

To join the current Lean results into a theorem from source to formal assembly, the next application proof must connect the translated C library to the Clight entry points used by the assembly certificates. A general source-translation theorem would also have to justify parsing and elaboration, including preprocessing and static-array handling. Reusing `CompCert`’s compiler theorem in Lean adds a different obligation: proving a sound interpretation of the `Rocq` semantics and results. A theorem about a native executable’s printed answer must also account for assembly printing, assembling, linking, and the output operation.

A Stewart’s 24 paragraphs

Appel and Bindel’s Section 2 follows the numbered paragraphs of Chapter 23 of Stewart [15]. The PDF abbreviates several paragraphs and skips 14; the accompanying `quadrature.v` comments include its root argument. The table compares their mathematical content with the `LeanPDE` revision used by this project. The general results below assume a compact interval, a positive continuous weight, and a positive node count. A proved result need not follow Stewart’s proof.

LeanPDE already supplied a normalized quadrature interface, the two-point Gauss–Legendre rule, polynomial exactness through degree three, affine transport to $[a, b]$, and stability under uniform perturbations. It also supplied the sharp estimate

$$\left| \int_a^b f(x) dx - Q_2(f) \right| \leq \frac{M(b-a)^5}{4320}, \quad |f^{(4)}(x)| \leq M,$$

under $a < b$ and $f \in C^4([a, b])$. On $[-1, 1]$ this is $M/135$. The theorem is `norm_integral_sub_gaussLegendreTwoRule_valueOn_1e` in LeanPDE’s `GaussLegendrePeano` module. Its Peano-kernel proof is preexisting work, distinct from the general Hermite remainder proved here. The companion [source comparison](#) gives exact declarations, file links, and dependency revisions.

Table 4: The 24 paragraphs, inherited results, and additions. Definitions and alternative arguments are identified explicitly; paragraphs 6–7 are not counted as separately formalized lemmas.

¶	Content	Already in LeanPDE	LeanQuadrature
1	Weighted quadrature sum	Finite normalized rules and their evaluation; no general weighted Gaussian construction.	✓ Defined: <code>GaussianRule</code> and its weighted sum on a compact interval.
2	Weighted integral notation	Uses <code>mathlib</code> ’s interval integral; the normalized rule targets unit weight.	✓ Defined: <code>polynomialIntegral</code> integrates a polynomial against w .
3	Linearity	Linearity of rule evaluation and <code>mathlib</code> integration.	✓ Inherited and specialized: the weighted polynomial integral is a linear map.
4	Orthogonality	No general quadrature inner product.	✓ Proved: <code>momentInnerCore</code> gives the polynomial inner product from a positive integral.
5	Orthogonal polynomial family	No general family in the quadrature development.	✓ Proved: the unique monic polynomial of each degree, with lower-degree orthogonality.
6	Expansion in a monic family	No corresponding quadrature lemma.	No separate basis-expansion theorem for an arbitrary monic family. Orthogonal projection supplies the construction directly.
7	Inductive expansion proof	No corresponding quadrature proof.	No separate proof of paragraph 6. <code>degree_sub_coeff_mul_lt</code> supplies degree reduction where the recurrence needs it.
8	Orthogonality to every lower degree	No general Gaussian-family theorem.	✓ Proved: <code>orthogonalPolynomial_orthogonal</code> for the constructed family.
9	The first two monic polynomials	Explicit two-point nodes, without this general family construction.	✓ Proved: <code>orthogonalPolynomial_zero</code> and <code>orthogonalPolynomial_one</code> .
10	Recurrence coefficients	No general recurrence coefficients.	✓ Proved: <code>recurrenceAlpha</code> , <code>recurrenceBeta</code> , and the recurrence with those coefficients.

¶	Content	Already in LeanPDE	LeanQuadrature (continued)
11	Vanishing lower terms	No general recurrence proof.	✓ Proved: the three-term identity has no further lower terms. The proof uses lower-degree orthogonality.
12	Three-term recurrence	No general recurrence theorem.	✓ Proved: <code>orthogonalPolynomial_recurrence</code> , with the positive norm-ratio identity.
13	Real, simple, interior roots	The two explicit nodes lie in the interval.	✓ Proved: splitting over $\mathbb{R}$, squarefreeness, and strict interior membership at every positive order.
14	Root argument	No general root argument.	✓ Alternative proof: factorization, constant sign, and positive integrals of squares give the root results.
15	Weights from Lagrange polynomials	General exactness from moments and an explicit two-point rule.	✓ Proved: <code>interpolatoryWeight</code> , interpolation exactness, and identification of Gaussian weights.
16	Degree $2n - 1$ exactness for n nodes	Two-point exactness through degree three.	✓ Proved: <code>gaussian_exact</code> uses polynomial division at every positive order.
17	Positive quadrature weights	The two weights are explicitly $1/2$ on $[0, 1]$.	✓ Proved: <code>gaussian_weight_pos</code> from the integral of a squared Lagrange polynomial.
18	Total weight and weight bound	Two-point total variation equals one on $[0, 1]$.	✓ Proved: <code>sum_weights</code> and <code>weight_le_mass</code> . The bound is $\int w$, already correct in the Rocq statement; the prose's bound of one requires normalization.
19	Gaussian remainder	The sharp two-point fourth-derivative bound above.	✓ Proved: first-derivative Hermite interpolation and the integrated $2n$ -derivative remainder, with explicit regularity.
20	Convergence for continuous functions	Uniform-perturbation bounds and composite two-point methods; no general limit as Gaussian order grows.	✓ Proved: <code>gaussian_rules_converge</code> . A separate theorem covers positive normalized rules of increasing exactness.
21	Gauss–Legendre specialization	The normalized two-point rule, affine transport, exactness, and its sharp error bound.	✓ Proved: the general compact theory specialized to unit weight; explicit rules through four nodes are identified with the monic family. Stored-table certificates cover ten orders.
22	Gauss–Laguerre on $[0, \infty)$	Outside this quadrature development.	◦ Open: no infinite-interval Gaussian construction or associated remainder and convergence proof.
23	Gauss–Hermite on $\mathbb{R}$	Outside this quadrature development.	◦ Open: no infinite-interval Gaussian construction. Hermite interpolation in paragraph 19 is a different subject.
24	Other choices of weight and interval	A concluding observation, not a theorem.	Our general result permits other positive continuous weights on compact intervals. It does not implement every named Gaussian family.

References

- [1] Reynald Affeldt, Yves Bertot, Alessandro Bruni, Cyril Cohen, Marie Kerjean, Assia Mahboubi, Damien Rouhling, Pierre Roux, Kazuhiko Sakaguchi, Zachary Stone, Pierre-Yves Strub, and Laurent Théry. MathComp-Analysis: Mathematical Components compliant analysis library. [Source repository](#), 2017.
- [2] Andrew W. Appel and David S. Bindel. Formalization of Gaussian quadrature and application verification (Preliminary Draft). Author manuscript. [Author PDF](#). The accompanying source notes identify the version examined, 2026.
- [3] Andrew W. Appel and David S. Bindel. The simple_cfem quadrature development. [Source repository](#), 2026.
- [4] Sylvie Boldo and Guillaume Melquiond. Flocq: A unified library for proving floating-point algorithms in Coq. In *20th IEEE Symposium on Computer Arithmetic (ARITH)*, pages 243–252, 2011.
- [5] Qinxiang Cao, Lennart Beringer, Samuel Gruetter, Josiah Dodds, and Andrew W. Appel. VST-Floyd: A separation logic tool to verify correctness of C programs. *Journal of Automated Reasoning*, 61(1–4):367–422, 2018.
- [6] CLean contributors. CLean: Clight semantics for Lean. [Source repository](#). Experimental port; local adaptations documented with the source.
- [7] Markus de Medeiros, Sergei Stepanenko, Zongyuan Liu, Oliver Soeser, Fernando Leal, Alvin Tang, Max Vistrup, Ralf Jung, Mario Carneiro, Joseph Tassarotti, Michael Sammler, and Lars Birkedal. Iris in Lean. [arXiv:2609.24252](#), 2026.
- [8] FloatLib contributors. FloatLib. [Source repository](#).
- [9] Robert Joseph. LeanQuadrature. [Project repository](#), 2026. The accompanying source archive contains the proofs discussed in this paper.
- [10] Ariel E. Kellison, Andrew W. Appel, Mohit Tekriwal, and David Bindel. LAProof: A library of formal proofs of accuracy and correctness for linear algebra programs. In *30th IEEE Symposium on Computer Arithmetic (ARITH)*, pages 36–43, 2023.
- [11] LeanPDE contributors. LeanPDE. [Source repository](#). Private dependency; access is available from the author on request.
- [12] Xavier Leroy. Formal verification of a realistic compiler. *Communications of the ACM*, 52(7):107–115, 2009.
- [13] Assia Mahboubi and Enrico Tassi. Mathematical Components. Zenodo, 2021.
- [14] Érik Martin-Dorel and Guillaume Melquiond. Proving tight bounds on univariate expressions with elementary functions in Coq. *Journal of Automated Reasoning*, 57:187–217, 2016.
- [15] G. W. Stewart. *Afternotes on Numerical Analysis*. Society for Industrial and Applied Mathematics, 1996.
- [16] The CompCert Project. The CompCert C verified compiler: General structure. [Official compiler manual](#).
- [17] The mathlib Community. The Lean mathematical library. In *Proceedings of the 9th ACM SIGPLAN International Conference on Certified Programs and Proofs (CPP)*, pages 367–381, 2020. [Source version used in this development](#).
- [18] The Verified Software Toolchain Project. Verified Software Toolchain. [Official project documentation](#).
- [19] The Verified Software Toolchain Project. Verified software units. [VST 2.16 source](#). Module composition, global initialization, and whole-program verification.